

E-Commerce App - Project Goal

This E Commerce Application project will be a complete full stack web platform built using Python, Django, PostgreSQL, Django REST Framework, Email Services, Docker, and GitHub Actions CI. The goal is to deliver one integrated application that demonstrates every topic covered in training through a real industry level system. The final submission will include a working website with user authentication, product management, cart and checkout flow, order processing, REST APIs, email notifications, database integration, container deployment, and automated CI pipeline.

The application must include a User Module where customers can register, login, manage profiles, and securely access their accounts using Django Authentication. Role based access must be implemented so that Admin users can manage products and orders while Customers can browse products and place purchases. Email based workflows such as signup confirmation and password reset must be supported through Django Email Service.

The Product Catalog Module must allow Admin users to perform complete CRUD operations on products and categories. Each product must have slug based SEO friendly URLs, and the frontend must display product listings with category filters, price sorting, and availability checks. Jinja2 templates with {Tailwind integrated UI} (**Optional**) must be used to build clean product pages and responsive layouts.

The Cart and Checkout Module must allow users to add products to cart, update quantities, remove items, and calculate totals dynamically. A checkout process must be included that converts cart items into an order. Proper exception handling must be implemented for cases like invalid quantities, missing products, or stock issues.

The Order Management Module must support order placement, order history for customers, and an Admin dashboard for tracking all orders. Orders must include status updates such as Pending, Shipped, and Delivered. Once an order is placed, an automated email confirmation must be sent to the user, and shipping updates must also trigger notifications.

A Payment Simulation Module must be implemented to demonstrate payment success and failure handling using dummy gateway logic. Payment status must be stored in the database, and error handling must be applied through try except blocks for reliability.

The project must include a complete Django REST Framework API layer exposing endpoints for products, cart actions, orders, and authentication. Serializers must be created for all models, and ViewSets with routers must provide structured REST APIs.

PostgreSQL must be used as the main database, integrated through Django ORM. Database creation and management should be demonstrated using tools like pgAdmin or DBeaver. The project must include ORM based queries for filtering, searching, and relational operations, along with basic raw SQL CRUD examples through command line usage.

The entire application must be deployment ready using Docker. The submission must include a Dockerfile for the Django application, a docker-compose.yml file connecting Django with PostgreSQL, and environment variable support for secure configuration. The project must run successfully in containers without manual dependency installation.

A CI pipeline must be implemented using GitHub Actions to automate testing, lint checks, and Docker build validation. The repository must contain a working workflow file under .github/workflows, and the pipeline must execute successfully on every push.

Python concepts must be clearly integrated throughout the project, including OOP based service classes such as CartManager, static methods for utilities, iterator or generator based pagination, safe demonstrations of shallow and deep copy in cart logic, and structured exception handling. Utility functions inspired by practice problems such as swapcase or split join can be included in helper modules to show strong Python foundation.

The final submission must include complete source code uploaded on GitHub, a professional README with setup instructions, database schema design, API documentation, UI screenshots, Docker deployment steps, and proof of CI pipeline execution. This single project submission will act as a complete capstone demonstrating Django web development, backend engineering, REST APIs, database handling, DevOps integration, and strong Python programming skills.

Build a full stack E Commerce web application using Python, Django, PostgreSQL, DRF, Email Service, Docker, and CI with GitHub Actions.

Core Deliverables

1. User Module

Requirements

- User Signup and Login using Django Authentication
- Role based access
- Customer
- Admin
- Profile management
- Password reset via email

Deliverables

- Auth pages with Tailwind UI
- Secure session handling
- Email verification workflow

2. Product Catalog Module

Requirements

- Product CRUD for Admin
- Product categories and subcategories
- Product slug based URLs
- Product listing with filters
- Price
- Category
- Availability

Deliverables

- Models for Product, Category
- Dynamic product pages using Jinja2 templates

- URL dispatcher with slugs

Example

```
/products/mobiles/iphone-15-pro
```

3. Cart and Checkout Module

Requirements

- Add to cart
- Update quantity
- Remove from cart
- Cart total calculation
- Checkout page

Deliverables

- Cart model linked to user
- Functional views and templates
- Exception handling for stock errors

4. Order Management Module

Requirements

- Place order from cart
- Order history for users
- Admin order dashboard
- Order status updates
- Pending
- Shipped
- Delivered

Deliverables

- Order and OrderItem models
- Admin panel integration
- Email confirmation after order placed

5. Payment Simulation Module

Requirements

- Dummy payment gateway logic
- Payment success and failure handling

Deliverables

- Payment status field
- Error handling with try except

6. Django REST Framework API Module

Requirements

Expose APIs for frontend or mobile integration

- Product API
- Cart API
- Order API
- Authentication API using token or JWT

Deliverables

- Serializers for all models
- ViewSets and Routers

Example Endpoints

```
GET /api/products/
```

```
POST /api/cart/add/
```

```
GET /api/orders/
```

7. Email Service Module

Requirements

Send emails for key actions

- Signup confirmation
- Password reset
- Order placed
- Order shipped

Deliverables

- SMTP integration
- Email templates
- Background safe email sending

8. Database and ORM Module

Requirements

Use PostgreSQL with Django ORM

- Schema design
- Relationships
- Admin database control

Deliverables

- Database setup via pgAdmin or DBeaver
- ORM queries for product search and filtering
- Raw SQL CRUD through command line

9. Deployment Ready Containerisation

Requirements

Dockerize the full application

- Django app container
- PostgreSQL container
- Environment variables support

Deliverables

- Dockerfile
- docker-compose.yml
- Setup documentation

10. CI using GitHub Actions

Requirements

Automated pipeline for

- Code linting
- Running tests
- Docker build validation

Deliverables

- .github/workflows/ci.yml file
- Build status badge

11. Python Concepts Integration

- Mandatory Usage
- OOP design for services like CartManager
- Static methods for utilities
- Iterators or generators for product pagination
- Shallow vs deep copy demo in cart logic
- eval usage avoided or secured example
- Practice style functions included in utilities

Deliverables

- utils.py with reusable functions
- Clean exception handling

Final Project Submission Deliverables

- Complete source code on GitHub
- README with setup steps
- Database schema diagram
- API documentation

- Screenshots of UI
- Docker deployment guide
- CI pipeline working proof
- Suggested Project Structure

```
ecommerce_app/  
  accounts/  
  products/  
  cart/  
  orders/  
  api/  
  templates/  
  static/  
  docker/  
  .github/workflows/  
  requirements.txt  
  manage.py
```